CyberLab: How to Use

A plain-language, step-by-step guide for the local production profile.

▼ Jump to a section (25 topics)

1. The simple explanation
2. What production means here
3. Install once, then start from anywhere
4. Keep the data from your earlier installation
5. Start automatically when you sign in
6. Understand the layout
7. Add a real project
8. Learn the controls with a sample
9. Run an investigation
10. Refresh current vulnerability and attack references
11. Set automatic refresh preferences
12. Check your dependencies against advisories
13. Use local AI with Ollama
14. Monitor a project
15. Read a change review
16. Review evidence and risk
17. Run verification deliberately
18. Check that the application is healthy
19. Back up, upgrade and recover
20. Troubleshooting
21. A practical daily routine
19. Set up your project passport (4.14)
20. Read your daily briefing
21. Check the versions in your npm lockfile
22. Recover an incomplete CVE refresh

Release **4.14.0**.

---

1. The simple explanation

Imagine your application is a house. CyberLab is your inspection notebook and reference library. It can list some of the doors, notice when the list changes, look up published safety problems, and record checks you approve.

Ollama is an optional assistant reading that notebook. It can suggest questions. It cannot decide that a door is unsafe just because it sounds suspicious.

The intelligence library receives new public safety bulletins. This is what “learning the latest vulnerabilities” means here: downloading, checking, and saving reference records with their source and dates. It does not change the model’s weights, invent zero-day knowledge, or watch every cyberattack happening worldwide.

---

2. What production means here

This release has a production HTTP server, bounded work queues, persistent storage, private file permissions, backups, a stable installer, and an optional user service. It is designed for one trusted user on a local Ubuntu computer.

The dashboard stays at `http://127.0.0.1:8780`. It is not an Internet-facing, multi-user website. Do not expose it through a public port forward. This profile does not provide organizational accounts, remote authentication, or a managed SOC.

No screen claiming “safe” or “confirmed vulnerability” is generated automatically. Production operation still needs a human to review applicability and test results.

---

3. Install once, then start from anywhere

Stop the earlier CyberLab terminal with Ctrl+C first. Download the current `cyberlab-ai.zip` and extract it into a fresh directory. Open a terminal inside the extracted `cyberlab-ai` directory.

1. Check that Python is available: `python3 --version`. Use Python 3.10 or newer.
2. Run the installer below. It creates a private runtime, verifies the packaged dependency, runs isolated smoke checks, then activates the release.
3. Use the stable start command it prints. You no longer need to remember the directory where the ZIP was extracted.
4. Open `http://127.0.0.1:8780` in your browser.

```
python3 scripts/install_production.py
```

The normal stable command is:

```
"$HOME/.local/share/cyberlab/start.sh"
```

If Ubuntu says venv or ensurepip is unavailable, install the Ubuntu package and rerun the installer:

```
sudo apt install python3-venv
```

Run the installer as your ordinary user, not with sudo. Its production dependency is bundled and hash-verified; installing it does not need a package-index download.

---

4. Keep the data from your earlier installation

Data is separate from program files. Replacing code does not mean replacing your notebook. Defaults remain `~/.cyberlab-v4` for evidence and `~/CyberLab-Projects` for projects.

If you previously used different folders, tell the installer those exact paths. The directory name “test” does not itself turn features on or off. The following example matches the earlier test workspace used in this conversation:

```
python3 scripts/install_production.py \
  --data-dir "$HOME/cyberlab-test-HBIgws/data" \
  --projects-dir "$HOME/cyberlab-test-HBIgws/projects"
```

1. In the old application, look at Settings and note its data and projects paths.
2. Stop the old process.
3. Install with those same paths if you want to retain that history.
4. Keep the old directories until the new app shows the records you expect.
5. Check the installer’s backup path. It creates a consistent SQLite backup before activation when an existing database is present.

You can instead choose fresh persistent folders for real work, leaving the old sample workspace untouched. Do not copy synthetic evidence into a real project and label it as a production observation.

---

5. Start automatically when you sign in

The optional user service starts CyberLab without keeping a terminal open:

```
python3 scripts/install_production.py --enable-service
```

Use the same data/projects flags from the previous section when installing into an existing custom workspace. Later installer runs preserve the saved paths unless you explicitly change them.

1. Stop a manually running copy before starting the service on the same port.
2. Run the installer with `--enable-service`.
3. Check its status with the command below.
4. Open the dashboard. The service does not automatically open a browser.

```
systemctl --user status cyberlab.service
systemctl --user stop cyberlab.service
systemctl --user start cyberlab.service
journalctl --user -u cyberlab.service -n 50 --no-pager
```

The user service normally starts at login. Running after logout depends on your Ubuntu user-manager/linger configuration; the installer does not change that system-wide setting. If no user service manager is available, use the stable launcher instead. A generated unit is not proof that the service started.

---

6. Understand the layout

The left sidebar is the tool cupboard. Each page has one main job. The project selector at the top is the label on your notebook: choose the project you want to inspect, then press Open. Navigation carries that choice with you. “All projects” gives a combined overview; tools that need a project will ask for one.

- **Overview:** the front cover. See projects, changes awaiting review, active monitors and recent activity. The numbers come from saved records, not a score that claims your application is safe.
- **Projects:** the filing cabinet. Open a local project or create a folder.
- **Investigations:** the flashlight. Choose a question about routes, changes, or a local application.
- **Monitoring:** the reminder clock. Compare route inventories on a schedule.
- **Intelligence:** the reference library. Refresh public bulletins and check npm dependencies against advisories.
- **Evidence:** the notebook. Review what ran, what it observed, and related records.
- **Verification:** the controlled test bench. Run only checks you explicitly approve.
- **How to Use:** this guide inside the application.
- **Settings:** the labels on the equipment. See storage paths, release and model configuration, and check the local Ollama connection.

7. Add a real project

1. Open Projects.
2. Enter a short project name, such as `my-app`.
3. Leave “Include a sample project” unchecked for a real project.
4. Confirm permission to create the local folder and press Create project.
5. Place the application files you intend to inspect in that folder. Settings shows the approved projects directory.
6. Open that workspace and confirm the project selector shows the correct name.

For current route tools, CyberLab recognizes Next.js `app/api`, `src/app/api`, `pages/api`, or `src/pages/api` filenames. For dependency intelligence it reads a `package.json` declaring npm dependencies. Other application types do not become supported simply because a folder was created.

Use an authorized working copy. Do not add credentials, private keys, real customer records, or an entire home directory. Filename inventory does not read arbitrary source contents. A symlink to a project outside the approved root is not listed.

8. Learn the controls with a sample

1. Open Projects and create `practice`.
2. Select “Include a sample project with synthetic routes.”
3. Authorize creation and press Create project.
4. Open its workspace.
5. Run Review project changes to establish its baseline.
6. Open Verification and choose Synthetic experiment.
7. Approve that sample check and press Run approved test.
8. Open Evidence to see the result.

The sample fixture deliberately contains an example mismatch. It is teaching data, not proof of a vulnerability. Creating a sample does not send live test requests.

9. Run an investigation

1. Open Investigations.
2. Confirm the active project.
3. Choose exactly one investigation type.
4. Read the action description and confirm authorization.
5. Press Start investigation.
6. Review the result or refresh its history if the job is still running.

Review project changes compares route filenames with the prior snapshot. The first run establishes a baseline. A later added or removed route creates a change review with links to both snapshots. Editing source text without changing filenames is not detected.

Discover application routes lists supported route filenames. It does not prove which HTTP methods or access-control rules the code implements.

Investigate with local AI asks Ollama to select allowed route metadata and suggest questions. Ollama cannot choose a shell command or an external target. When a root-proven package check links an advisory to cached CVE references, bounded reference context can also be supplied. A reference is background, not proof that the project is affected.

Inspect a local test application is a bounded local-target investigation. Use an explicit loopback URL such as `http://127.0.0.1:3000/` and only an application you own or are authorized to inspect. Remote targets are rejected.

---

10. Refresh current vulnerability and attack references

1. Open Intelligence.
2. Look at the three source cards.
3. Check Last successful refresh, Publisher update, and freshness state.
4. To update now, approve downloading the public feed and press Refresh this source.
5. Wait, then reload the page to see the new count and status.
6. Use the search box below the cards to look up a CVE ID, product name, or technique.
7. Open a result and follow its official source link before deciding what it means.

The sources have different meanings:

- **CISA KEV:** vulnerabilities known to have been exploited somewhere. Default refresh interval: one hour. An entry is a reason to check affected products, versions and vendor guidance; it is not a finding against your project.
- **CVE Program recent disclosures:** the publisher's latest delta window, refreshed every five minutes. At most 20 record documents are processed per refresh. Observed records beyond that per-refresh limit are retained in a persistent queue and processed on later refreshes. This is not a complete historical CVE mirror: changes can still be missed while offline or between publisher windows.
- **MITRE ATT&CK:** a catalog of methods attackers use, refreshed daily. This is reference knowledge, not a live feed proving attacks on your computer. The initial enterprise catalog is a large download, approximately 55 MB at validation.

The two vulnerability feeds and attack catalog are downloaded from their official GitHub publisher repositories. Source text is treated as untrusted data. CyberLab does not visit arbitrary links embedded in feeds or download exploit programs.

NVD ingestion is not part of this release. A cloud connectivity probe to the NVD API was denied; it must not be represented as a successful integration.

---

11. Set automatic refresh preferences

Public intelligence refresh starts automatically when the installed application is running, unless it was installed with `--no-auto-intelligence`.

1. Open Intelligence.
2. Expand Automatic refresh preferences on a source card.
3. Check or uncheck Automatically refresh this source.
4. Choose its interval in seconds: 300 is five minutes; 3600 is one hour; 86400 is one day. Allowed range: five minutes to seven days.
5. Approve the preference change and save it.

Pausing a source does not delete its cached records. An already-started network request may finish; a changed setting prevents stale work from replacing the current generation's cache. Refreshing public feeds does not enable scheduled Ollama analysis or active application verification.

If the last refresh fails, the last good records remain available. "Stale" means that you need a newer successful check, not that the underlying bulletin is false. Check coverage text: even a fresh CVE delta is a limited window.

---

12. Check your dependencies against advisories

1. Confirm your project has a valid `package.json`.
2. Select that project and open Intelligence.
3. Scroll to Check project advisories.
4. Read the data-sharing description: dependency names and versions are sent to OSV.
5. Approve the request and press Check advisories.
6. Open its history record after it finishes.
7. Review each candidate against the actual installed version and vendor guidance.

A range such as `^4.17.0` is not the installed version. Such matches remain unresolved. An exact declared version still needs configuration and lockfile review. A linked CISA KEV reference raises review priority; it does not confirm exploitation. Details fetching is bounded to 25 advisories and a total time budget. Some candidate details may be unavailable; do not interpret that as "no vulnerability."

---

13. Use local AI with Ollama

1. Start your installed Ollama service.
2. Ensure the model shown in Settings exists locally.
3. Open Settings and press Check connection.
4. Model available means the model name was listed. It does not yet prove that the model can follow CyberLab's recommendation schema.
5. Run the optional live smoke command below from the installed current application.
6. Ask for an AI investigation only after that check succeeds.

```
cd "$HOME/.local/share/cyberlab/current"
.venv/bin/python scripts/smoke_test.py --live-ollama
```

The default model is `llama3.2:latest`. If you choose to install it, the normal Ollama command is `ollama pull llama3.2:latest`; downloading a model uses disk space and bandwidth. The installer does not download or start Ollama for you. To change the saved model, rerun the installer with `--model YOUR_MODEL_NAME`.

"NOT RUN: real Ollama" in the offline smoke output means the optional check was skipped. It does not tell you whether Ollama is stopped. Connection unavailable means a real connection or response problem occurred. Model suggestions stay unverified even when a live smoke test passes.

14. Monitor a project

1. Select a project and open Monitoring.
2. Use Compare routes now first to establish its baseline.
3. Enable monitoring and choose a check interval.
4. Leave scheduled Ollama analysis off unless you specifically want it.
5. Authorize and save preferences.
6. Open a change review when new route metadata appears.
7. Uncheck Enable monitoring and save to pause.

User jobs that have not started can be cancelled during shutdown; rerun them after restarting if needed. The observed CVE backlog remains saved. Monitoring runs while the CyberLab process is running. It does not continue after you stop that process. A running user service can keep the process alive while its user manager remains active. The source refresh schedule is separate from the project monitoring schedule.

15. Read a change review

1. Open Monitoring and select a Change review.
2. Review the added, removed or metadata-changed route list.
3. Open the current and previous assessment links under Evidence sources.
4. If desired, approve Request analysis / retry to ask local Ollama for a suggestion.
5. Read the hypothesis, reason, confidence label and source links.
6. Decide whether to review code manually or separately prepare an approved test.

The technical record preserves input context. Later observations appear separately so that saved model input does not silently change. Public references are identified as library records, not project observations. Their links pin a saved record revision, so a later publisher change or rejection does not erase the original reference. Hallucinated evidence IDs and unknown actions are rejected rather than executed.

16. Review evidence and risk

1. Open Evidence and confirm the project.
2. Use All activity, Observations, Risk reviews or Evidence memory to focus the list.
3. Open a record to see its original observations.
4. Use Analyze relationships to correlate exact-route evidence.
5. Use Build memory to save a snapshot of related observations.

An AI hypothesis is a suggested question. A synthetic observation is fixture data. A localhost observation is something a controlled local request returned. A risk indicator is a reason for investigation. None alone establishes a confirmed flaw. After an unexpected exit, a previously running job may be marked as an interrupted error. Its partial observations remain saved; review them and authorize a new action if needed. It is not automatically replayed. Legacy records with unknown project roots remain viewable but are excluded from new root-scoped model context.

17. Run verification deliberately

Synthetic experiment: an offline teaching or regression check using `cyberlab-test-fixture.json`. It sends no live HTTP traffic.

Localhost verification: up to 12 bounded GET cases from `cyberlab-live-plan.json`, with synthetic account labels and an approved loopback application. No cookies, passwords, production accounts, redirects, or remote scans are permitted by this tool.

1. Review your fixture or plan before using it.
2. For localhost checks, start the separate test application first.
3. Select the correct project and open Verification.
4. Choose the appropriate test type.
5. Explicitly approve that particular set of checks.
6. Run it and review the history record.
7. Investigate a mismatch against the intended access policy; do not label a single HTTP status as a proven security vulnerability.

A model recommendation cannot grant permission to run a test.

---

18. Check that the application is healthy

Open the health endpoint locally:

```
python3 - <<'PY'
import json, urllib.request
with urllib.request.urlopen('http://127.0.0.1:8780/healthz',timeout=5) as response:
    print(json.load(response))
PY
```

Expected fields include `status: ok`, the current release, `transport: waitress`, and `database: ok`. This is process/database health, not proof that a threat feed or Ollama works. Check those on Intelligence and Settings separately.

---

19. Back up, upgrade and recover

1. Keep backups outside the active application code directory.
2. Note your data and projects paths in Settings.
3. Stop the old process before starting the new one on the same port.
4. Extract a fresh release and run its production installer.
5. The installer backs up an existing SQLite database, stages a new runtime, verifies the dependency hash, and runs smoke checks before switching its stable current pointer. The previous code release remains available.
6. Start the new process or enable/restart the managed user service.
7. Confirm your projects and evidence are present before removing any old download.

To roll back, stop CyberLab first. Preserve newer evidence separately, restore a chosen pre-upgrade database backup to a separate recovery directory, and use it with the preserved previous code. Never overwrite a live database. A pre-upgrade backup does not contain observations collected after that backup.

20. Troubleshooting

File not found: you are probably in your home directory rather than the source folder. After installation, use the printed stable start command from anywhere.

Port already in use: stop the old manual process or managed CyberLab service. The installer does not kill an unknown application using the port.

Another process uses this data directory: do not run two CyberLab processes against the same database. Stop the first one before retrying.

Public source refresh failed: check Internet access and the source status card. Required public downloads use `raw.githubusercontent.com`; package checks use `api.osv.dev`. Previous good data remains available. Do not disable TLS verification.

No matching references: wait for the first refresh, reload the page, clear the reference type filter, or try a specific CVE/product term. No match is not a safety certificate.

Too many jobs / action already running: wait for the current action and refresh its history. Repeated clicks are deliberately deduplicated.

Old layout: confirm the release in Settings and press Ctrl+Shift+R in the browser.

Sandbox bwrap error: this concerns the local coding agent's execution sandbox. A working Ubuntu terminal can still run CyberLab. Changing the application does not repair that agent sandbox; do not run the coding agent as root as a workaround.

---

21. A practical daily routine

1. Start CyberLab or check its user service is running.
2. Confirm the active project and the freshness of the public reference library.
3. Review new project changes.
4. Check dependency advisories after dependency updates.
5. Ask local AI for a focused suggestion when useful.
6. Review the source evidence and approve only the specific verification you need.
7. Record your conclusions and keep a current database backup.

CyberLab helps organize defensive work. Human judgment, vendor patches, normal testing and independent review are still necessary.

---

19. Set up your project passport (4.14)

Think of the passport as the label on your inspection notebook.

1. Select your project in **Active project** at the top and click **Open**.
2. Click **Project passport** in the sidebar.
3. Enter the responsible person/team and explain what the application does.
4. Describe who should access which information. Do not paste passwords or keys.
5. Choose development, staging, or production and the sensitivity of the data.
6. Confirm the save checkbox and click **Save passport**.

7. Review the persistent ID, canonical folder and dependency coverage on the left.

These notes are declarations, not verified security controls or permission for tests. Each canonical folder has a stable ID. Preserve its path during upgrades. Renaming or moving it makes a new identity; automatic relinking is not implemented. Older records whose root was never captured remain in all-workspace history, but CyberLab will not guess ownership from a shared folder name.

---

20. Read your daily briefing

Think of this as the notebook's contents page, with next steps.

1. Keep a project selected and click **Overview**.
2. Read **Your daily briefing**. It is assembled when you open the page.
3. Follow **Review evidence** for a route change, dependency check, linked CISA reference, or intelligence-source problem.
4. If coverage says unknown, run an approved advisory check. If a check is stale, repeat it after reviewing dependency changes.
5. If a candidate is linked to CISA KEV, prioritize manual investigation. Known exploitation elsewhere does not prove your application can be exploited.

This is deterministic saved-evidence reporting. It is not a continuous detector, AI training, an inbox of every new global attack, or a security clearance.

---

21. Check the versions in your npm lockfile

A manifest is the shopping list. A lockfile is the receipt for resolved versions. The receipt still does not prove what is running on the production server.

1. Put `package.json` and its matching `package-lock.json` or `npm-shrinkwrap.json` in the selected project root. CyberLab never installs or executes packages.
2. Open **Intelligence**, approve the external package lookup, and run the check.
3. Refresh the advisory history and open the completed check.
4. Read **Dependency coverage** before interpreting any matches: queried/total pairs, lockfile source, manifest consistency and unresolved entries.
5. Review each candidate's exact version and source. Multiple versions of the same package stay separate, including transitive packages.
6. Independently verify deployed versions, affected code and reachability before treating a candidate as an actionable vulnerability.

Supported npm lockfile formats: 1, 2 and 3. Shrinkwrap takes precedence. Format 1 cannot establish manifest consistency. Workspaces, links and unresolved entries are reported as incomplete. Files over 4 MB or inventories over 3,000 entries are rejected. Checks query up to 1,000 unique pairs in direct-first order; details show query coverage and the first 100 inventory rows. Without a supported lockfile, manifest-only results explicitly retain version uncertainty. No pnpm/yarn parser, source AST analysis or deployed dependency verification is provided in this release.

---

22. Recover an incomplete CVE refresh

1. Open **Intelligence** and find the CVE Program source card.
2. Read the queued and quarantined counts. Good records remain searchable.

3. A failed individual record automatically retries after five then ten minutes. The third failure quarantines it so other downloads continue.
4. After connectivity or the publisher document is corrected, approve and click **Retry failed CVE records**, then refresh the page after the job finishes.
5. A newer publisher revision also resets that record's failure state.

“Fresh” never means all historical CVEs were downloaded. Partial status means records remain unresolved, even if the delta index itself downloaded successfully.